

# The Ingestion Plane

I'll continue with the next installment. Note: in my numbering this is **Part 4 — the Ingestion Plane** (installment 2 of the white paper), the connector-by-connector engineering deep-dive I promised. Let me firm up a few precise technical specifics first. Good — I now have authoritative specs for the Raindrop, Claude Code, and Codex formats. Two connectors (X/Twitter and Perplexity) need fresh confirmation for proper citation before I write them up. #WHITE PAPER — COMPOUND (continued)

## Part 4 — The Ingestion Plane

*Installment 2 of the COMPOUND white paper. Part 1–3 established the thesis, the reverse-engineered VSI parity spec, and the four-plane/five-primitive architecture. This installment specifies the Ingestion Plane connector-by-connector — the largest delta between base VSI and COMPOUND, because it is where the founder's own tool exhaust becomes primary signal.*

---

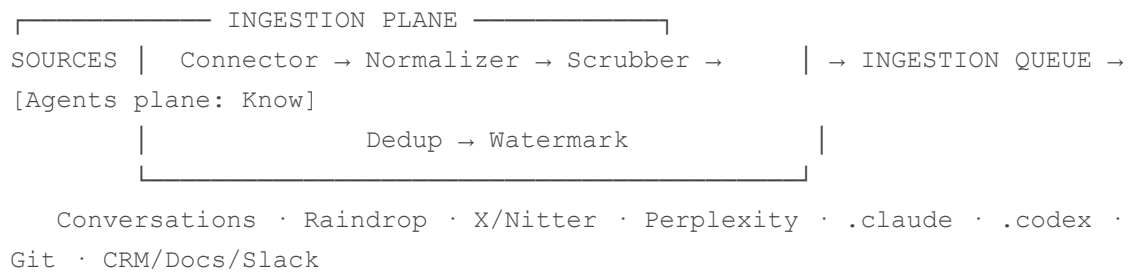
### 4.0 Plane overview & the ingestion contract

The Ingestion Plane has exactly one job and one prohibition. **Its job:** pull deltas from every source, normalize them into the universal Thread/Episode/Signal shape, and enqueue them for the Agents plane. **Its prohibition:** it never writes to Company Memory directly and never reasons over content — that is the Agents plane's monopoly (see Part 3, §3.1, the trust-boundary rule). This separation is what keeps the audit trail total: every fact in memory can be traced to an ingestion event with a content hash and a dual timestamp.

Every connector, regardless of source, must satisfy the same six-clause **ingestion contract**:

1. **Idempotency** — re-ingesting the same source item produces no duplicate Episode. Dedup key = `sha256(source_type + native_id + content_hash)`.
2. **Dual timestamping** — emit both `t_event` (when it happened in the world) and `t_ingest` (when COMPOUND saw it). The temporal-graph memory (Part 5) depends on this.
3. **Provenance completeness** — author/handle/URL/commit/file-path captured or the item is quarantined, not dropped.
4. **Incremental cursors** — never full-scan; persist a per-source watermark (timestamp, ID, ETag, or git SHA) and pull only what's new.
5. **Secret-scrubbing before persistence** — credentials, tokens, `.env` contents, and PII are redacted at the connector boundary (critical for coding-session sources, §4.6–4.7).
6. **Graceful degradation** — a source that fails must fail *loudly* (alert + health flag), never silently. This clause exists specifically because of the Twitter/Nitter failure mode (§4.4).

Text



## 4.1 The normalization pipeline (Source → Thread → Episode → Signal)

Before the connectors, the shared shape. Everything funnels into four layers:

- **Source** — a configured account/endpoint (one Raindrop account, one monitored X list, one machine's `~/codex`).
- **Thread** — an append-only stream from a single source *instance* (one Raindrop collection, one X handle, one coding session, one research session). This is VSI's "live thread" primitive (Part 3, §3.2).
- **Episode** — a single ingestible chunk within a Thread (a bookmark + its highlight, one tweet, one coding-session turn, one research answer). Episodes are the unit of idempotency and the unit the temporal graph stores (Part 5, §3.4).
- **Signal/Claim/Decision/Question/Contradiction** — the extracted atoms (Screen G taxonomy). Extraction happens in the **Know** step (Agents plane), *not* in ingestion — ingestion only produces Episodes.

The connector's deliverable is therefore a stream of **normalized Episode records**:

```
{}
```

 JSON

```
{
  "episode_id": "uuid",
  "thread_id": "raindrop:collection:1234",
  "source_type":
  "raindrop|x|perplexity|claude_session|codex_session|git|conversation|...",
  "native_id": "source-native-identifier",
  "t_event": "2026-05-29T22:14:03Z",
  "t_ingest": "2026-05-30T02:14:31Z",
  "actor":
  {"kind": "founder|model|external", "handle": "...", "display": "..."},
  "content": { "...source-specific normalized body..." },
  "links": ["https://..."],
  "provenance":
  {"url": "...", "path": "...", "commit": "...", "instance": "..."},
  "content_hash": "sha256:...",
  "scrub_report": {"redactions": 3, "types": ["api_key", "email"]},
  "raw_ref": "cold-storage-pointer"
}
```

Now, each connector.

---

## 4.2 Conversations connector (VSI parity — Zoom / Meet / Teams)

This is the one COMPOUND inherits wholesale from VSI's Ingest row (Screen D). It is summarized here for completeness; it is not the novel part.

- **Mechanism:** bot/recall-style join or post-call webhook → transcript + diarized speaker turns + recording artifact.
- **Thread = one meeting.** Episode = one speaker turn (or a coherent turn-group).
- **Provenance:** speaker identity is first-class — VSI's evidence rows attribute claims to roles like "VP Sales" / "Sr. BD" (Screen C), and COMPOUND preserves the same speaker→role mapping so a Claim can carry "who said it."
- **Cadence:** event-driven (webhook on call end) + a nightly reconciliation sweep for missed webhooks.

The reason this matters as the baseline: conversations are the *internal* store of the three-store split (Part 3, §3.4 — *internal* / *market* / *decided*). Everything in §4.3–§4.8 below mostly fills the **market** and **build** stores, which base VSI under-serves.

---

## 4.3 Raindrop bookmarks connector — “what the founder is paying attention to”

Bookmarks are a high-signal, low-noise proxy for a founder’s evolving interests, and Raindrop exposes a clean, well-documented REST API plus an official MCP server — the easiest of the new connectors.

**Endpoint & auth.** The base URL is `https://api.raindrop.io/rest/v1/`, all request/response bodies are JSON, timestamps are ISO 8601, and the rate limit is 120 requests per minute per authenticated user.<sup>3</sup>(<https://explainx.ai/skills/intellectronica/agent-skills/raindrop-api>) COMPOUND uses OAuth2 in production: the authorization code is exchanged via POST to `https://raindrop.io/oauth/access_token` for an `access_token` and `refresh_token`, with `expires_in` given in seconds.<sup>2</sup>(<https://developer.raindrop.io/v1/authentication/token>) The operational catch to design around: access tokens expire after two weeks and must be refreshed with the `refresh_token` grant.<sup>10</sup>(<https://eliteai.tools/agent-skills/raindrop-api>) COMPOUND therefore runs a token-refresh daemon that rotates well before the two-week boundary; for single-user dev/test, a non-expiring test token from the app settings is supported via the same `Authorization: Bearer` header.

**Rate-limit handling.** Responses carry `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset` (UTC epoch seconds), and exceeding the limit returns HTTP 429.<sup>1</sup>(<https://developer.raindrop.io/>) The connector reads these headers and self-throttles; on 429 it backs off to the reset time. Because pagination uses `perpage` and `page` parameters with a maximum of 50 items per page,<sup>5</sup>(<https://playbooks.com/skills/intellectronica/agent-skills/raindrop-api>) a full backfill of a large account loops pages while respecting the 120/min ceiling — comfortably within budget for nightly deltas.

**What we pull.** The API exposes Collections, Raindrops, Highlights, User, Tags, Filters, Import, Export, Backups, and an MCP server.<sup>6</sup>(<https://developer.raindrop.io/v1/authentication>) COMPOUND maps these as:

- **Thread = one Collection** (e.g., “Pricing experiments,” “Competitor teardown”). Collection structure mirrors the founder’s own mental taxonomy — free Knowledge-Tree branches (Screen E).
- **Episode = one Raindrop (bookmark) + its highlights.** The Highlights endpoint is the high-value part: a highlight is the founder explicitly flagging a *specific passage*, which is far stronger signal than the bare URL. COMPOUND attaches highlights and notes to the bookmark Episode.
- **Tags** become candidate entity labels; **Filters** are used for cheap delta-detection — the filters endpoint returns aggregated counts for broken links, duplicates, favourites, untagged items, tags, and content types,<sup>3</sup>(<https://explainx.ai/skills/intellectronica/agent-skills/raindrop-api>)

[skills/raindrop-api](#)) which lets the connector detect “new untagged items since last run” without paging the whole collection.

**Backfill & resilience.** For initial onboarding COMPOUND ingests history two ways: API paging, and the bulk path — periodically exporting collections to HTML or CSV via the backup/export endpoints,<sup>5</sup>(<https://playbooks.com/skills/intellectonica/agent-skills/raindrop-api>) which is also the disaster-recovery snapshot. (The same `POST /import/file` path that imports a browser bookmark HTML file<sup>5</sup>(<https://playbooks.com/skills/intellectonica/agent-skills/raindrop-api>) lets a founder seed COMPOUND from a non-Raindrop bookmark manager on day one.)

**Cadence:** every 30–60 min (bookmarks are bursty but not real-time-critical) plus the nightly **Learn** sweep.

**Routing** → new/highlighted bookmarks become **interest-drift Signals** feeding Auto-Research (Screen A: ICP/pricing/channel sweeps) and the Positioning branch. A cluster of bookmarks on, say, usage-based pricing is exactly the kind of pre-conscious founder signal that should nudge the Pricing Model Agent’s next sweep.

---

## 4.4 Twitter / X connector — the one fragile link (Nitter + mandatory fallback)

Your brief specifies “twitter feeds daily via nitter.” I’ll build that, but the architecture must be honest about a hard 2026 reality, because clause 6 of the ingestion contract (fail loudly) exists primarily for this source.

**The reality.** In 2026 the primary official Nitter instance and many long-standing public instances have transitioned to a decommissioned state, because the upstream platform implemented aggressive rate-limiting and mandatory authentication for nearly all data requests.<sup>1</sup>(<https://blog.thefix.it.com/is-nitter-still-working-the-definitive-2026-status-report/>) RSS specifically is degraded: nitter.net RSS returns a blank page, xcancel.com requires a special User-Agent, and on other instances RSS feeds are blocked by bot challengers.<sup>4</sup>(<https://github.com/zedeus/nitter/issues/1353>) The mechanism that breaks it: Nitter relies on “account pools” — collections of credentials used to fetch data — and when those accounts are flagged and suspended by the primary service, the instance ceases to function until new credentials are provided.<sup>1</sup>(<https://blog.thefix.it.com/is-nitter-still-working-the-definitive-2026-status-report/>)

The reliability literature is blunt about why this can’t be the *only* path: the problem isn’t that self-hosted scrapers never work — it’s that they work inconsistently, then fail at the worst time,<sup>8</sup>(<https://stepper.io/blog/rss-for-twitter-feed>) and tracking has shown over 50% downtime over 12 months for many popular endpoints, creating a reliability gap for automation pipelines.<sup>8</sup>(<https://stepper.io/blog/rss-for-twitter-feed>) The operating rule COMPOUND adopts

verbatim: if a Twitter RSS method depends on someone else's unofficial scraper staying alive, you don't have a system — you have a temporary convenience.<sup>8</sup>(<https://stepper.io/blog/rss-for-twitter-feed>)

**The design: a provider-abstraction interface with a health-checked pool and a paid fallback.**

COMPOUND's X connector is a `TwitterProvider` interface with three ranked implementations, tried in order, with automatic failover:

| Tier                   | Implementation                                                                              | When used   | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|---------------------------------------------------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 (preferred for cost) | <b>Self-hosted Nitter</b> in Docker with the founder's own session token                    | Healthy     | Self-hosting via Docker is currently the most effective way to make Nitter work, because a private instance doesn't attract the same automated scrutiny as a high-traffic public one; <sup>1</sup> ( <a href="https://blog.thefix.it.com/is-nitter-still-working-the-definitive-2026-status-report/">https://blog.thefix.it.com/is-nitter-still-working-the-definitive-2026-status-report/</a> ) requires a real account token since guest accounts no longer work. |
| 1                      | <b>Public-instance pool via a rotating proxy</b> , modeled on <code>nitter-rss-proxy</code> | Tier 0 down | This pattern iterates through multiple Nitter instances until it finds a working one, and rewrites GUIDs/links to point at the canonical twitter.com URL so the reader doesn't show duplicate items from different instances <sup>7</sup> ( <a href="https://codeberg.org/derat/nitter-rss-proxy">https://codeberg.org/derat/nitter-rss-proxy</a> ) — exactly the dedup behavior COMPOUND needs.                                                                    |

|                    |                                             |                                                   |                                                                                                                                                                                                                                                                  |
|--------------------|---------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 (production SLA) | <b>Paid X API / commercial scraping API</b> | Tiers 0–1 unhealthy, or feed is decision-critical | The most durable fix for technical teams is a custom API-based pipeline;8( <a href="https://stepper.io/blog/rss-for-twitter-feed">https://stepper.io/blog/rss-for-twitter-feed</a> )<br>COMPOUND routes any feed flagged “decision-critical” straight to Tier 2. |
|--------------------|---------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

A **health monitor** polls each instance/provider on a schedule (the public Nitter status trackers are themselves a model for this) and demotes unhealthy providers; because public instances are now intermittent, an instance-health monitor is required just to find a functional gateway.1(<https://blog.thefix.it.com/is-nitter-still-working-the-definitive-2026-status-report/>)  
The connector emits a `feed_health` metric to the Governance plane so a founder is never silently flying blind on a competitor’s timeline.

**What we pull.** Thread = one monitored handle, list, or search query. Episode = one post (with quote/reply context). Daily cadence per your brief ( `x feeds daily via nitter` ), with decision-critical handles polled hourly.

**Routing** → posts become **market-store Signals** feeding the Competitor Scanner (Screen A) and Drift Detection (a competitor announcing usage-based pricing is precisely the kind of external signal that should challenge a standing Position). Because of the fragility, X is weighted *lower* in evidence scoring than first-party sources (an E2–E3 ceiling unless corroborated — see Part 5’s E1–E5 ladder).

---

## 4.5 Perplexity & research-thread connector — “what the founder is actively investigating”

Research threads are the founder reasoning *out loud against the live web*. They are dense with Claims and Questions and, critically, arrive **pre-cited**, which makes them the highest-yield input to Evidence Scoring.

**Two ingestion modes:**



**(a) COMPOUND-initiated research (the Auto-Research engine's web arm).** The **Reflect** step's combinatorial sweep (Screen A) needs live web grounding, and Perplexity's API supplies it. The Sonar family spans four tiers — a lightweight model for quick factual lookups, a pro model for complex multi-source queries, a reasoning model with chain-of-thought, and a deep research model that conducts exhaustive multi-step searches<sup>5</sup>(<https://techjacksolutions.com/ai-tools/perplexity/perplexity-api-guide/>) (with `sonar-pro-search` and `sonar-reasoning-pro` also exposed). COMPOUND tiers its calls to control cost:

- *Cheap fact-checks / drift confirmation* → `sonar` .
- *Multi-source synthesis for a hypothesis* → `sonar-pro` .
- *A full overnight market/competitor report* → `sonar-deep-research` , which autonomously searches hundreds of sources to deliver expert-level analysis.<sup>8</sup>(<https://www.mindstudio.ai/models/sonar>)

For the overnight cycle the **async mode** is essential: Sonar Deep Research offers an asynchronous API designed for complex research tasks, letting you submit a job and poll for completion<sup>2</sup>(<https://community.perplexity.ai/t/new-async-mode-for-sonar-deep-research/424>) — so COMPOUND dispatches deep-research jobs at the start of the nightly cycle and harvests them later in the same run rather than blocking. The Search API (distinct from the chat models) is used for raw retrieval: it returns ranked results with domain filtering, multi-query search, and content extraction.<sup>4</sup>(<https://www.perplexity.ai/api-platform>) Domain and recency filters let COMPOUND scope a sweep to, e.g., competitor domains or the last 30 days.

The cost-governance reason this is tiered and budgeted: you don't directly control how many searches run — the model decides, and a single Deep Research query can trigger dozens of searches,<sup>3</sup>(<https://www.cloudzero.com/blog/perplexity-api-pricing/>) on top of base token pricing plus citation tokens and reasoning tokens.<sup>3</sup>(<https://www.cloudzero.com/blog/perplexity-api-pricing/>) COMPOUND enforces a per-cycle research budget cap (a direct response to the agent-cost blowout warning in Part 1, §1.3) and logs spend per hypothesis so the Auto-Research leaderboard (Screen A) can show cost-adjusted scores.

**(b) Founder-conducted research threads.** When the founder runs their own Perplexity/other-tool research, COMPOUND ingests those transcripts (via export/import, or API where the tool offers thread retrieval). Thread = one research session; Episode = one query→answer turn. The decisive advantage over a raw web bookmark: every answer the Sonar models return comes with URLs pointing to the web sources that informed it, collapsing the LLM-call-plus-separate-search pipeline into a single call<sup>5</sup>(<https://techjacksolutions.com/ai-tools/perplexity/perplexity-api-guide/>) — so each research Episode arrives with its citations attached, and the **Know** step can promote those to **Claims with E-scores keyed to the cited source's authority**, not just to the model's assertion.

**Note for procurement (governance plane):** Perplexity has no public compliance statement specific to EU AI Act GPAI requirements as of mid-2026, so for regulated workflows this should be verified directly before deployment.<sup>7</sup>(<https://suprmind.ai/hub/perplexity/pricing/>) COMPOUND records the model + provider on every research Episode so the Decision Audit Trail can later answer “which model produced this Claim.”

**Routing** → research Episodes feed Auto-Research, Evidence Scoring, and the Questions backlog (Screen G's `Questions 28`) — open questions a founder is researching are exactly the system's to-do list.

---

## 4.6 `.claude` coding-session connector — mining the founder's build reasoning

This is the most novel and most technically involved connector, and the payoff is large: a founder's `.claude` sessions contain not just code but the *reasoning, decisions, blockers, and architecture choices* behind the build. A local agent on each founder's machine (or CI runner) tails these files and ships normalized, **scrubbed** Episodes.

**Where & what.** Claude Code stores every session as a JSONL file — one JSON object per line — under `~/.claude/projects/<encoded-project-path>/<session-id>.jsonl`.<sup>2</sup>(<https://claude-dev.tools/docs/jsonl-format>) Each file is a chain of typed message records linked by `parentUuid`: user prompts, assistant responses with content blocks (text, tool calls, thinking), tool results, system prompts, summaries, and git snapshots.<sup>1</sup>(<https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>) The connector parses each line by its `type` discriminator: top-level fields include `type`, `uuid`, `parentUuid`, `timestamp`, `sessionId`, `cwd`, `gitBranch`, and `version`, and inside `message.content` are content blocks of type `text`, `thinking`, `tool_use` (with `id`, `name`, `input`), and `tool_result` (referencing `tool_use_id`).<sup>2</sup>(<https://claude-dev.tools/docs/jsonl-format>) Token accounting is read from `message.usage` with `input_tokens`, `output_tokens`, `cache_creation_input_tokens`, and `cache_read_input_tokens` <sup>2</sup>(<https://claude-dev.tools/docs/jsonl-format>) so COMPOUND can attribute build-cost per project.

Crucially, this is a *complete* record, not a summary: the full message-by-message record includes tool calls with exact inputs and outputs, extended thinking blocks, subagent spawning events, token usage per turn, model selection, working directory, and git-state snapshots.<sup>1</sup>(<https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>)

**Thread/Episode mapping.** Thread = one logical conversation; Episode = one message record. But “one logical conversation” ≠ “one file”, which is the parsing trap. The connector must reconstruct conversations the way Claude Code itself does: it reads all the JSONL files in the project directory, parses every line into one flat list, and reconstructs the DAG from `uuid` and `parentUuid`.<sup>6</sup>(<https://piebald.ai/blog/messages-as-commits-claude-codes-git-like-dag-of-conversations>) Two structural facts drive the reconstruction logic:

- **Compaction boundaries.** When context approaches the limit (~167K tokens in practice), Claude Code writes a `compact_boundary` record whose `parentUuid` is null and whose `logicalParentUuid` references the last message before compaction.<sup>3</sup>(<https://blog.fsck.com/releases/2026/02/22/claude-code-session-continuation/>)
- **Cross-file continuation.** A continuation file’s first record points its `parentUuid` at the parent session’s last record, and both files share the same human-readable `slug`,<sup>3</sup>(<https://blog.fsck.com/releases/2026/02/22/claude-code-session-continuation/>) which COMPOUND uses as the stable conversation-level key.

**Dedup is mandatory.** Per the idempotency clause and a documented quirk: Claude Code sometimes writes the same message (same UUID) to multiple JSONL files during branching or resumption, so token counting without deduplication gives inflated totals — dedup by UUID before summing.<sup>1</sup>(<https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>) COMPOUND dedups by `uuid` at ingest.

**Subagent / team reconstruction.** Modern Claude Code spawns subagents (and, in team mode, a lead + specialists), each in its own file. COMPOUND rebuilds the execution tree from the metadata fields `parentToolUseId` (the tool call that spawned a subagent), `agentId`, `agentType`, and `teamName`.<sup>1</sup>(<https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>) This matters because your brief is explicitly about founders “steering multi-agent systems” — COMPOUND must represent the *whole agent team’s* work as one coherent build Thread, not 7 orphaned files.

**Beating auto-deletion (a real risk).** Claude Code is not an archive: recent sessions are kept intact, older sessions may have transcripts compacted or trimmed, and very old sessions are eventually removed from disk<sup>4</sup>(<https://claude-world.com/tutorials/s16-session-storage/>) (community cleanup scripts routinely delete histories older than 30 days<sup>5</sup>(<https://gist.github.com/gwpl/e0b78a711b4a6b2fc4b594c9b9fa2c4c>)). COMPOUND therefore does **not** rely on the files persisting. The local agent:

1. **Tails in near-real-time** (filesystem watch on `~/.claude/projects/`), shipping Episodes as lines are appended (the format is append-only, so this is safe). Claude Code writes new lines as the session progresses; nothing is rewritten or deleted within a session file.<sup>2</sup>(<https://claude-dev.tools/docs/jsonl-format>)

2. **Hooks the lifecycle** using Claude Code's `PreCompact` and `SessionEnd` hooks to force a capture *before* compaction discards detail and before cleanup removes old files.
3. **Correlates file edits** — file-history-snapshot records are correlated with file-write tool calls to produce a clean diff of what a session actually modified<sup>1</sup>(<https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>) — so a build Episode records *what changed*, not just *what was discussed*.

**Secret-scrubbing (contract clause 5, non-negotiable here).** Coding sessions are saturated with secrets — `.env` dumps, API keys printed by tool calls, connection strings. The connector runs a redaction pass (regex + entropy detection + known-key-prefix matching) on every `tool_use.input`, `tool_result`, and `text` block **before** the Episode leaves the machine, and records a `scrub_report`. Raw content never transits unredacted.

**Routing** → coding-session Episodes feed the **Build** store and the Product/Roadmap branch (Screen E). Extracted atoms: architecture **Decisions** (“chose Postgres over Mongo because...”), **Blockers/Questions, Signals** about velocity, and — uniquely — **Reflections** (Screen F's fifth memory type), since the `thinking` blocks are literally the founder's/agent's reasoning. This is how COMPOUND's memory learns *how the founder builds*, not just *what they decided*.

---

## 4.7 `.codex` coding-session connector — the same discipline, a different on-disk format

Founders mix tools, so COMPOUND ingests OpenAI Codex sessions with a parallel connector. The conceptual mapping is identical to §4.6; only the file format differs.

**Where & what.** Codex CLI stores all session transcripts as JSONL files under `~/.codex/sessions/YYYY/MM/DD/`, each saved as a `rollout-*.jsonl` file containing the full conversation history, tool calls, and token usage.<sup>7</sup>(<https://inventivehq.com/knowledge-base/openai/how-to-resume-sessions>) The precise path is `~/.codex/sessions/YYYY/MM/DD/rollout-TIMESTAMP-UUID.jsonl`.<sup>1</sup>(<https://deepwiki.com/openai/codex/3.5.2-rollout-persistence-and-replay>) The line structure is one level more nested than Claude's: each line is a `RolloutLine` that wraps a `RolloutItem` and adds a UTC timestamp,<sup>1</sup>(<https://deepwiki.com/openai/codex/3.5.2-rollout-persistence-and-replay>) and the `RolloutItem`'s `type` field selects the payload. The connector reads:

- `SessionMeta` — the payload for `session_meta` entries — containing `id`, `timestamp`, `cwd`, `cli_version`, `model_provider`, and `git` info (session identity, working directory, git context),<sup>6</sup>(<https://pkg.go.dev/github.com/thesimonho/warden/agent/codex>) where the

git block supplies `git_sha`, `git_branch`, and `git_origin_url`.<sup>1</sup>(<https://deepwiki.com/openai/codex/3.5.2-rollout-persistence-and-replay>) This `git_origin_url` is gold: it's the join key linking a coding session to the reference-repo connector (§4.8) and thus to the actual artifact.

- `TokenCountInfo` (cumulative `total_token_usage` and per-turn `last_token_usage`) with input/cached/output/reasoning/total counters<sup>6</sup>(<https://pkg.go.dev/github.com/thesimonho/warden/agent/codex>) — and the parser must treat these as cumulative, forwarding them directly rather than re-accumulating.
- Tool use and assistant output via `response_item` entries.

**Discovery & indexing.** COMPOUND mirrors Codex's own index strategy: Codex uses a SQLite database ( `state.sqlite` ) as the primary query layer for session history, and on startup performs a backfill by scanning rollout files to populate it.<sup>1</sup>(<https://deepwiki.com/openai/codex/3.5.2-rollout-persistence-and-replay>) The connector reads this SQLite index for fast “what's new since watermark” discovery, falling back to a date-directory glob. (Session titles come cheaply: the title is derived from the first `UserMessage`.<sup>1</sup>(<https://deepwiki.com/openai/codex/3.5.2-rollout-persistence-and-replay>))

**The persistence-policy trap (must handle explicitly).** Codex does *not* write everything to disk by default, which means a naïve parser silently loses the most important reasoning. Codex's rollout persistence policy filters which events land in JSONL: in limited mode (the CLI default), begin-events ( `exec_command_begin`, `mcp_tool_call_begin`, `patch_apply_begin` ), approval/permission requests, elicitation requests, and stream errors are never persisted.<sup>6</sup>(<https://pkg.go.dev/github.com/thesimonho/warden/agent/codex>) Tool use is captured via `response_item` entries (always persisted), but error details from end-events require extended persistence mode, which is only available via `codex app-server` (the `persist_extended_history` parameter).<sup>6</sup>(<https://pkg.go.dev/github.com/thesimonho/warden/agent/codex>) **COMPOUND's mitigation:** for founders who opt in, the local agent runs Codex under `codex app-server` with extended persistence so error/approval context is captured; where that's not possible, COMPOUND flags affected Episodes as “limited-fidelity” so downstream extraction doesn't over-trust an incomplete record.

**Data-loss resilience.** Same philosophy as Claude: don't trust the files to survive. There is a documented failure where only one `rollout-*.jsonl` remained on disk while ~90 thread IDs in the index pointed to missing rollout files — durable chat payloads removed without metadata cleanup, orphaning history.<sup>5</sup>(<https://github.com/openai/codex/issues/21196>) COMPOUND's near-real-time tail + ship means an Episode is safely in the queue long before any local cleanup or corruption. And because Codex has no long-term memory across sessions by default (its separate “Memories” feature

aside),<sup>8</sup>(<https://www.verdent.ai/guides/codex-cli-resume-continue-save-chat>) COMPOUND becomes that cross-session memory — exactly the role the white paper assigns to Company Memory.

**Bonus integration:** Codex can run as an MCP server over stdio so other tools can connect to it,<sup>3</sup>(<https://developers.openai.com/codex/cli/reference>) which dovetails with COMPOUND's own MCP surface (Part 3, §3.2) — Codex can both *emit* sessions to COMPOUND and *read* Company Memory back through MCP during a build.

**Routing** → identical to §4.6: Build store, Roadmap, architecture Decisions, Reflections — unified with Claude sessions under a single per-project build Thread keyed by `git_origin_url` + repo.

---

## 4.8 Reference-repo (Git) connector — grounding sessions in actual artifacts

Coding sessions describe intent; repos hold the result. The Git connector ties them together and is the backbone of the Build step and the Clone Engine (Part 8).

- **Mechanism:** clone/pull each configured reference repo; subscribe to push webhooks for real-time deltas; periodic full re-index for code-graph drift.
- **Thread = one repository.** Episode types: a **commit** (message + diff stat + author + SHA), a **PR/issue**, and a **structural snapshot** (dependency graph, module map, public API surface).
- **The join.** The `git_origin_url` / `git_sha` / `git_branch` captured from `.codex SessionMeta` (§4.7) and the `gitBranch` /git-snapshot records from `.claude` (§4.6) let COMPOUND stitch “*this reasoning produced this commit produced this artifact*” into a single causal chain — the provenance spine that makes the Decision Audit Trail (Governance plane) actually complete.
- **Reference repos as clone templates.** For the Legacy-Business Cloning Engine, the connector also ingests *external* reference repos (open-source implementations of the legacy model's components) into a **pattern library** — the raw material from which the Clone Architect assembles an agent-run replacement (Part 8).
- **Secret-scrubbing:** the same redaction pass runs over diffs and commit bodies; committed secrets are flagged to Governance as a security finding, not memorized.

**Routing** → Build store, Product/Roadmap, and the Clone pattern library.

---



## 4.9 Standard SaaS connectors (CRM · Notion · Slack · Email · Docs) — parity, in brief

These complete VSI's Ingest row (Screen D) and Screen G's 8-source pipeline. They're well-trodden, so only the COMPOUND-specific routing is noted:

- **CRM** → Decisions and pipeline Signals; the *decided* store's commercial reality-check against the *internal* store's optimism (drives Contradiction detection).
- **Notion / Docs** → Claims and Positions (formal write-ups); often the canonical text a Position document points to.
- **Slack / Email** → Signals, Questions, and informal Decisions; high-noise, so scored lower in the E-ladder unless corroborated.

All five follow the same contract (incremental cursors via each API's change-token/`updated_since`, dedup by native ID, scrub PII).

---

## 4.10 Cross-cutting machinery (scheduler · dedup · scrubbing · backpressure · idempotency)

The connectors share one runtime:

- **Scheduler.** A per-source cadence registry: event-driven where webhooks exist (conversations, Git), short polls for bursty sources (Raindrop 30–60 min), **daily** for X per your brief, and a master **nightly Learn sweep** that reconciles every source against its watermark (catches missed webhooks and fragile-source gaps).
- **Watermarks.** Per-source cursors persisted transactionally with the enqueue, so a crash never double-ingests or skips: Raindrop (last-seen timestamp + collection ETag), X (last post ID per handle), Perplexity (job IDs + thread cursor), `.claude` / `.codex` (file offset + last `uuid`/rollout line + `slug`), Git (last SHA per branch).
- **Global dedup.** `sha256(source_type + native_id + content_hash)` in a dedup store, with the source-specific UUID dedup for Claude layered on top.
- **Secret-scrubbing service.** Centralized redaction (regex + entropy + key-prefix dictionaries) applied at the boundary for *all* sources, mandatory for coding sessions and Git; emits `scrub_report` to Governance.
- **Backpressure & cost guards.** The ingestion queue applies per-source rate caps (Raindrop's 120/min, Perplexity's per-cycle budget) and sheds/queues load rather than melting a downstream API. Perplexity spend is metered per hypothesis (\$4.5).
- **Idempotent replay.** Because every Episode is content-addressed and the raw payload is kept in cold storage (`raw_ref`), the entire **Know** extraction can be re-run on historical Episodes when the extraction model improves — without re-hitting any source API.

## 4.11 Failure-mode & reliability matrix

| Source                | Primary risk                                  | Likelihood  | COMPOUND mitigation                                                                | Residual evidence ceiling                 |
|-----------------------|-----------------------------------------------|-------------|------------------------------------------------------------------------------------|-------------------------------------------|
| Conversations         | Missed webhook                                | Low         | Nightly reconciliation sweep                                                       | E5 (first-party, attributed)              |
| Raindrop              | Token expiry (2-week)                         | Med         | Refresh daemon; test-token fallback; CSV export DR                                 | E4                                        |
| <b>X / Twitter</b>    | <b>Instance/feed death</b>                    | <b>High</b> | <b>3-tier provider failover + health monitor + paid fallback; fail loudly</b>      | <b>E2-E3 (corroboration required)</b>     |
| Perplexity            | Cost blowout / model opacity                  | Med         | Per-cycle budget cap; tiered models; record model+provider                         | E3-E5 (inherits cited source's authority) |
| <code>.claude</code>  | Auto-deletion / compaction loss               | Med-High    | Near-real-time tail + <code>PreCompact / SessionEnd</code> hooks                   | E4                                        |
| <code>.codex</code>   | Limited-mode persistence gaps; file orphaning | Med-High    | <code>app-server</code> extended persistence; tail-and-ship; flag limited-fidelity | E4                                        |
| Git                   | Committed secrets                             | Med         | Scrub + security finding to Governance                                             | E5 (artifact = ground truth)              |
| CRM/Docs/Slack /Email | API change                                    | Low-Med     | Change-token cursors; schema-version guards                                        | E3-E5 by source                           |

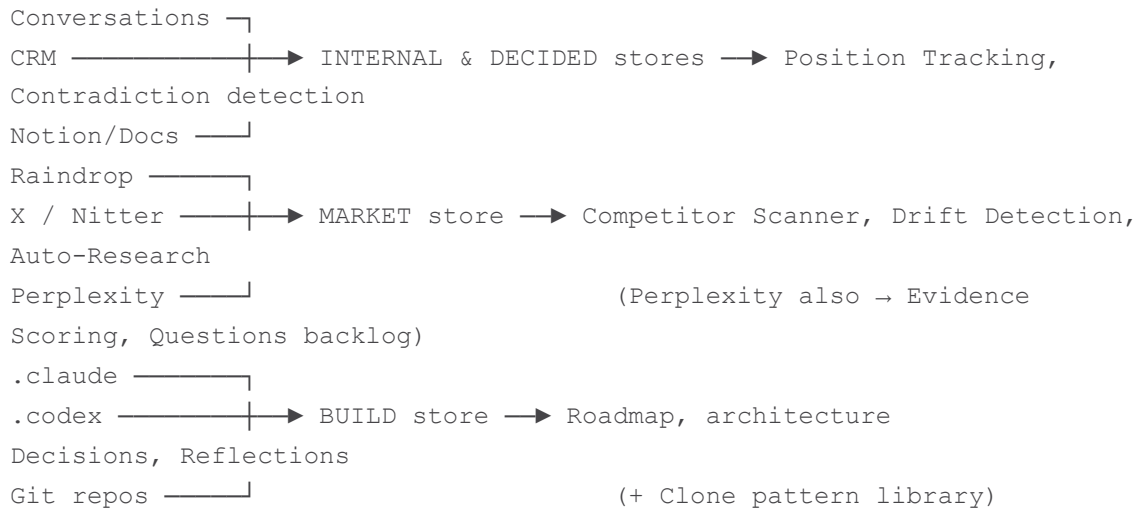
The single most important row is X: it is the only source whose *default* state in 2026 is “broken,” which is why its connector carries the most machinery and the lowest trust weight.



---

## 4.12 Routing summary — what feeds what

Text



Every arrow terminates at the **Know** step of the nightly loop (Part 3, §3.3), where Episodes become scored Signals/Claims/Decisions/Questions/Contradictions and enter the temporal knowledge graph.

---

## End of installment 2 (Part 4)

That fully specifies the Ingestion Plane: the six-clause contract, the Source→Thread→Episode→Signal normalization, and engineering-grade connectors for conversations, Raindrop, X/Nitter (with its mandatory fallback architecture), Perplexity/research threads, `.claude`, `.codex`, Git, and the standard SaaS sources — plus the shared scheduler/dedup/scrubbing/backpressure runtime and a failure matrix.

**Next installment (Part 5 — Company Memory)** will detail the temporal knowledge graph that everything above flows into: the bi-temporal `t_valid` / `t_invalid` edge model, the three-store separation (internal / market / decided) and how Contradiction detection becomes a single cheap query across them, the E1–E5 evidence-scoring ladder (formalizing Screen C's distribution chart), Position objects with confidence-history (Screen E), and how Drift Detection is implemented as an edge-invalidation event.

Say **continue** for Part 5 — or, if you'd rather I jump ahead, tell me and I'll deliver **Part 8 (the Legacy-Business Cloning Engine)** next instead, since that's the most ambitious part of your brief.

---

Learn more:

1. [Overview | API Documentation - Raindrop.io](#)
2. [Inside Claude Code: The Session File Format and How to Inspect It | by Yi Huang | Medium](#)
3. [Rollout Persistence and Replay | openai/codex | DeepWiki](#)
4. [Is Nitter Still Working? The Definitive 2026 Status Report](#)
5. [Models - Perplexity](#)
6. [Obtain access token | API Documentation](#)
7. [Claude Code JSONL transcript format explained](#)
8. [Session/Rollout Files · openai/codex · Discussion #3827](#)
9. [Nitter Instance Health](#)
10. [New Async Mode for Sonar Deep Research! - Announcements - Perplexity API Platform Forum](#)
11. [raindrop-api — AI agent skill | explainx.ai | explainx.ai](#)
12. [How Claude Code Session Continuation Works](#)
13. [Command line options – Codex CLI | OpenAI Developers](#)
14. [Nitter Alternatives 2026: View Twitter \(X\) Timelines Anonymously | Simple Web](#)
15. [Perplexity API Pricing In 2026: Models, Costs, And Optimization Tips](#)
16. [API Reference - Raindrop](#)
17. [Session Storage - ClaudeWorld](#)
18. [Features – Codex CLI | OpenAI Developers](#)
19. [RSS feeds dont work in nitter.net and xcancel.com · Issue #1353 · zedeus/nitter](#)
20. [Perplexity API Platform — AI Search & Grounded LLM APIs for Developers](#)
21. [raindrop-api skill by intellectronica/agent-skills - playbooks](#)
22. [Claude Code `--continue` after Directory `mv` move - Migration Guide and Internal Mechanics explanation · GitHub](#)
23. [Data loss: resumed-thread errors due missing rollout JSONL files \(state still has threads, files gone\) · Issue #21196 · openai/codex](#)
24. [Twitter rss solution - idea - The NewsBlur Forum](#)
25. [Perplexity API & Developer Guide: Search-Grounded AI From Setup to Production \(2026\)](#)
26. [Authentication | API Documentation](#)
27. [Messages as Commits: Claude Code's Git-Like DAG of Conversations – Piebald Blog](#)

28. [codex package - github.com/thesimonho/warden/agent/codex - Go Packages](#)
29. [Why not try a Nitter instance. It offers RSS and none of the URLs are shortened ... | Hacker News](#)
30. [Sonar Deep Research API Pricing 2026 - Costs, Performance & Providers](#)
31. [Raindrop](#)
32. [claude-code-data-structures.md](#)
33. [codex resume: How to Resume Your Last Codex CLI Session | Inventive HQ](#)
34. [derat/nitter-rss-proxy: HTTP server that proxies Twitter RSS feeds served by Nitter - Codeberg.org](#)
35. [Perplexity Pricing 2026: Free, Pro, Max, Enterprise, and Sonar API Costs - Suprmind](#)
36. [Raindrop.io API — ClawHub](#)
37. [GitHub - withLinda/claude-JSONL-browser: A web-based tool that converts Claude Code CLI conversation logs \(JSONL format\) into human-readable Markdown. Features a built-in file explorer for managing multiple logs at once. · GitHub](#)
38. [Codex CLI Resume, Continue, and Save Chat Explained - Verdent Guides](#)
39. [RSS for Twitter Feed: A 2026 Guide to Reliable Feeds | Free AI Workflow Automation Software](#)
40. [Sonar — AI Model | MindStudio](#)
41. [raindrop-api | Skills Marketplace](#)
42. [GitHub - HillviewCap/clog: A web-based viewer for Claude Code conversation logs with real-time monitoring and cross-platform support. · GitHub](#)
43. [CLI flag to save trajectory/output as JSON for non-interactive `codex exec` runs · Issue #2288 · openai/codex](#)
44. ["If nothing changes, all remaining Nitter instances will go down eventually" - Hacker News](#)
45. [Perplexity AI Ultimate Guide 2026: Features, Pricing, API, and Complete Walkthrough · AI Tools DevPro](#)
46. [raindrop-api - AI Agent skill](#)
47. [ClaudeCode.History — ClaudeCode v0.36.3](#)
48. [Free, Unlimited Perplexity AI API](#)